Certicom-131 benchmark: Pollard rho and index calculus on this repository's code

Study `research/certicom131_rho_ic_bench_20261022/`, commit `02b37807a` plus this directory. Run window 2026-10-22. Every number below is labelled **measured**, **counted** or **derived** (a product of recorded numbers; the formula is given). Raw records are in `raw/`; `results.json` holds every value used here.

Summary

- **Counted, exact group orders.** Expected rho work is 2^60.81 steps for ECC2K-130 (class size 262, negation x Frobenius), 2^64.83 for ECC2-131 and 2^64.92 for ECCp-131 (class size 2). Against the published 2^60.9 for ECC2K-130 the ratios are **15.2x** (ECC2-131) and **16.2x** (ECCp-131); against the audit's own 2^60.81 they are 16.2x and 17.3x. The table's "about 15x" holds for ECC2-131. Its "about 21x" for ECCp-131 does not follow from iteration counts alone: the remaining factor of roughly 1.2 to 1.3 would have to be per-iteration cost on the target GPU, which this host cannot measure.
- **Measured, one pinned core of this Xeon (2.1 GHz, KVM, no GPU).** ECC2K-130 production client: **13.2 M iterations/s** (5 runs, 11.4 to 14.0). ECC2-131 with a pclmulqdq field and batched-affine negation walk (batch 128): **8.50 M steps/s**. ECCp-131 with the repository's 32-bit Montgomery arithmetic (batch 128): **1.23 M steps/s**. Sharing one inversion across 128 walks cuts the step cost by **20x (ECC2-131 pclmul)** and **24x (ECCp-131)**: the batched-inversion idea carries over to both new targets, measured.
- **Derived.** At these rates the expected rho work is about 4,800 core-years (ECC2K-130), 122,000 core-years (ECC2-131) and 899,000 core-years (ECCp-131) on this host class. The repository's recorded RTX PRO 6000 rate (17.09 B/s, not measured here) gives 3.99 GPU-years at 2^60.9, matching the table's 3.9. No CUDA path exists yet for ECC2-131 or ECCp-131, so no GPU figure is given for them.
- **Index calculus, measured.** On the repository's own ladders (all 24 instances verified) the best variant costs **3.4x to 9.4x** the matched rho's S on prime curves (2^10 to 2^24), **11x to 24x** on random binary curves (2^12 to 2^26) and **2.0x to 75x** on Koblitz curves from 2^16 to 2^39, with a 161x outlier at one instance. Below 2^11 on Koblitz it is under 1x where rho's setup dominates. No instance larger than 2^39 was run, and the repository's regime rule keeps full-width runs of all three targets out of reach (48-bit end-to-end envelope).
- **Exact ECCp-131 relation probe, measured.** 25 isolated runs on the published curve found **0 natural relations and rank 0** every time (two-point factor bases 256, 512, 1024; three-point base 128). The per-candidate hit probability is bounded by 2^-113.2, 2^-111.2, 2^-109.2 and 2^-106.2 respectively (derived from the repository's support bound). It is a bounded diagnostic, not a solver.
- **Classification (AGENTS.md section 3).** This study is a baseline measurement. It changes no code under test, claims no improvement, and adds no leaderboard row. Distances to the matched rho are reported as distances.

Sources checked and how they inform this work

- Bernstein, Chen, Cheng, Lange, Niederhagen, Schwabe, Yang, "Breaking ECC2K-130", ePrint 2009/541 (PDF text read in full for the passages used). It states that walks on orbits of size 2 x 131 reduce the iteration count by that factor, that its iteration function raises the work to sqrt(pi l / (2 x 2 x 131)) x 1.069993, about 2^60.9 iterations, and that a Core 2 core ran 529 cycles per iteration (22.45 M iterations/s on four cores). I use 2^60.9 as the published reference and the audit's 2^60.81 (no walk penalty) as the counted baseline; both ratios are shown. Wiener-Zuccherato and Gallant-Lambert-Vanstone (the orbit argument) are cited by that paper and not re-read here.
- Repository: `ecc2k130/README.md` and `ecc2k130/RTX-PRO6000.md` (recorded GPU rates), `problems/certicom-binary-gpu-rho-integration.md` (ECC2-131 and ECCp-131 CUDA replay still open), `problems/eccp131-full-parameter-index-calculus.md` and `benchmarks/20261021-eccp131-exact-probe/` (prior Apple M4 probes; I reproduce the protocol on x86), `docs/ic/LEADERBOARD.md` (pre-gate rows; the same direction as the ladders here), `icx estimate` output for the three curves (regime "scaled").
- Not reproduced, named as missing (AGENTS.md section 15): the published 2009 per-core rates are the only external rho figure; no external tool was run on ECC2-131 or ECCp-131, and no fastest-published comparison exists for them. Marked "external comparison: not run".

Host, instruments, protocol

Host (recorded): Intel Xeon @ 2.10 GHz, 4 vCPU, 15 GiB, KVM guest, no PMU, no GPU, Linux 6.18, rustc 1.97.0, g++ 13.3. Timed commands ran on reserved CPU 3 through `tools/isolated_bench.py` (48 records, 1 flagged contended: `nist_bench_eccp-131_r1`, 0.1 CPU-s from my own `git` call, excluded and never pooled; 1 run, `k130cpu_1t_r3`, refused by the gate for the same reason, re-run as `r3b`). Expect 5 to 10% residual noise on a VM; the A/A pair below measures it.

Instruments and gates: (a) `ecdlp-nist audit` for the exact expected work; (b) `ecdlp-nist bench`, generic Rust big-integer arithmetic, 5 runs; (c) `ecc2k130/ecc2k130-cpu --bench`, the production ECC2K-130 client, 1 thread, 5 runs, plus one unisolated 4-thread run; (d) `bench131.cpp`, new here: an r-adding walk with the negation map (16 jumps, jump chosen from low bits of x) on the exact ECCp-131 and ECC2-131 groups using `gpu/certicom131/certicom131.cuh`, one inversion per batch by Montgomery's trick, batches 1, 8, 32 and 128, 7 interleaved rounds. Gates, all passed: every end point on the curve; end states identical for every batch size; for ECC2-131 the pclmulqdq field is checked against the header's shift-xor field on 20,000 random product and square cases and 200 inversions (0 mismatches) and its walk reproduces the header arm's trajectories; (e) `ic prime --curve eccp-131`, five interleaved rounds per arm with a duplicated B512 arm as A/A; (f) `ic boundary --seed 131` default ladders, all 24 instances verified, 594 s.

Unpriced work (AGENTS.md section 5a): the walk benchmark times the field arithmetic, partition hash and negation canonicalisation of a step. It does not carry the jump-scalar bookkeeping, the distinguished-point test, or any server-side storage and collision work; those are unpriced on every rho arm and are reported as missing, not as zero. The cross-method ratios below are the ones the repository's own `ic boundary` emits (its pinned calibrations); the 0 / 0.1 / 1 GAE band was **not** re-run here, so no ranking between index calculus and rho is claimed beyond "above the reference at every measured size".

Results: rho

curve	class size	log2 expected steps (counted)	ratio vs ECC2K-130 (2^60.81)	ratio vs 2^60.9
ECC2K-130	262	60.809	1.0	0.94
ECC2-131	2	64.826	16.2	15.2
ECCp-131	2	64.919	17.3	16.2

Table 1: Expected rho steps from `ecdlp-nist audit` on the exact published group orders. Ratio is derived: 2 to the power of the difference of the log2 columns.

arm (one core)	batch 1 ns	8	32	128	steps/s at 128
ECC2-131, pclmulqdq field	2373	384	171	118 (96 to 123)	8.50 M
ECCp-131, 32-bit Montgomery	19549	2944	1198	813 (755 to 907)	1.23 M
ECC2-131, header shift-xor field	403995	60371	26155	16351	61 k
ECC2K-130 production client					13.2 M (11.4 to 14.0)
generic Rust: ECC2K-130 / ECC2-131 / ECCp-131					127 k / 140 k / 57 k

Table 2: Measured median ns per walk step by inversion batch size (min to max in brackets where shown); rounds: 7 (5 for the header arm, which ran 20 steps). Field costs on this host: pclmul multiply 20.5 ns, inversion 2.0 us (98 multiplications); 32-bit Montgomery multiply 120 ns, inversion 19.9 us (165 multiplications). The generic-Rust figures are noisy (spread up to 45%; ECCp-131 samples 36.5 k to 66.0 k) and are L0 descriptive only.

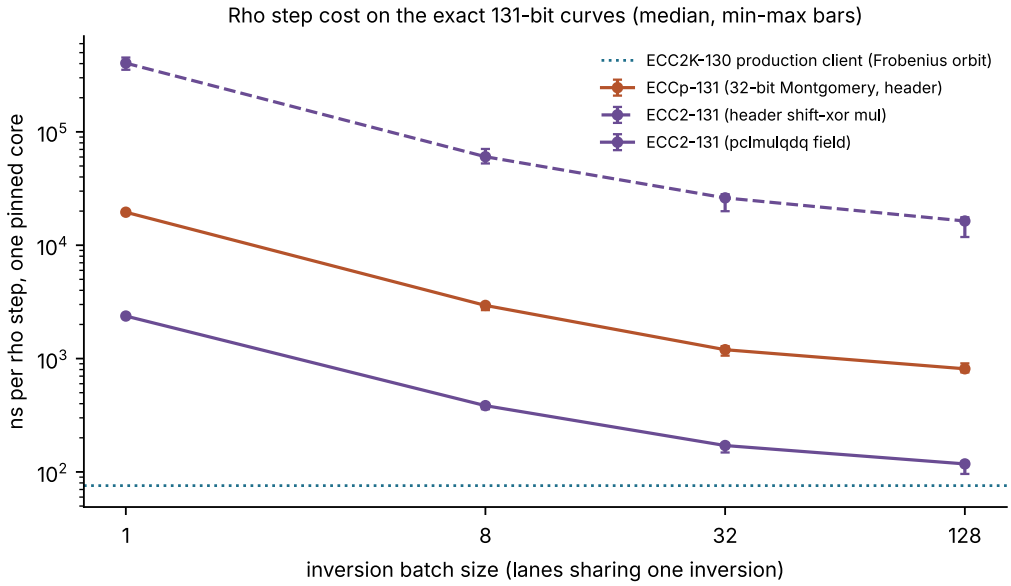


Figure 1: Measured ns per rho step against inversion batch size, one pinned core. Run `raw/bench131_*.stdout`.

curve	arm	core-years (derived)	vs ECC2K-130 client
ECC2K-130	production client, 13.2 M/s	4,847	1.0
ECC2-131	pclmul batch 128, 8.50 M/s	121,936	25.2
ECCp-131	32-bit Montgomery batch 128, 1.23 M/s	898,526	185

Table 3: Derived: 2^(log2 expected steps) / rate / 31,557,600 s. Different implementations on each row; the ECC2K-130 client is a tuned bitsliced Frobenius-orbit walk, the others are straightforward batched affine walks. Differences between rows mix algorithm and engineering and are not an implementation ranking.

Checks against the table in the request. ECC2K-130 in GPU-years from recorded repository rates (derived, not measured here): 3.99 at 17.09 B/s (RTX PRO 6000) and 5.45 at 12.50 B/s (RTX PRO 4500), at 2^60.9. The table's 3.9 corresponds to the first. For ECC2-131, 15.2x on iterations gives about 61 GPU-years only if its per-step GPU cost equalled the Koblitz kernel's; on this CPU my ECC2-131 step costs 1.55x the ECC2K client's iteration, so the ratio is unverified for the GPU. For ECCp-131 the claim that "only the scheduling ideas carry over" is consistent with the measurement: batched inversion carries over (24x measured); the carry-less multiply path does not apply to a prime field.

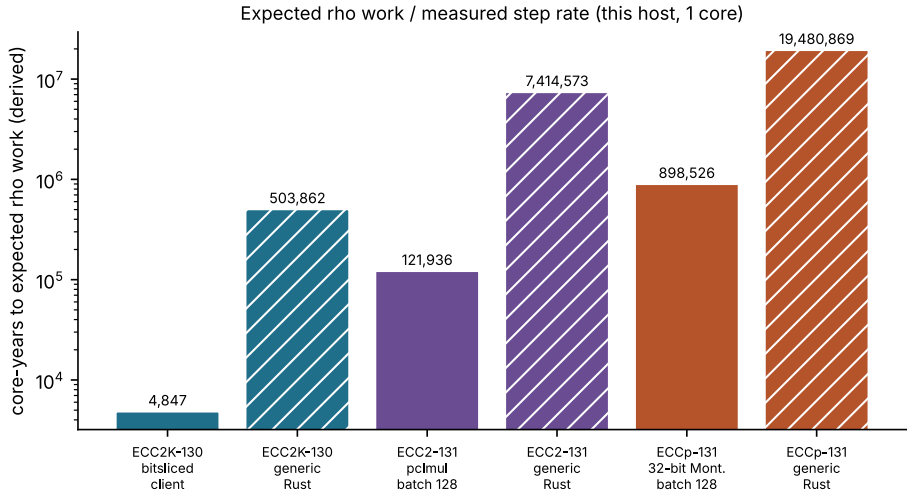


Figure 2: Derived core-years to the expected rho work from the measured rates (hatched: generic Rust arithmetic).

Results: index calculus

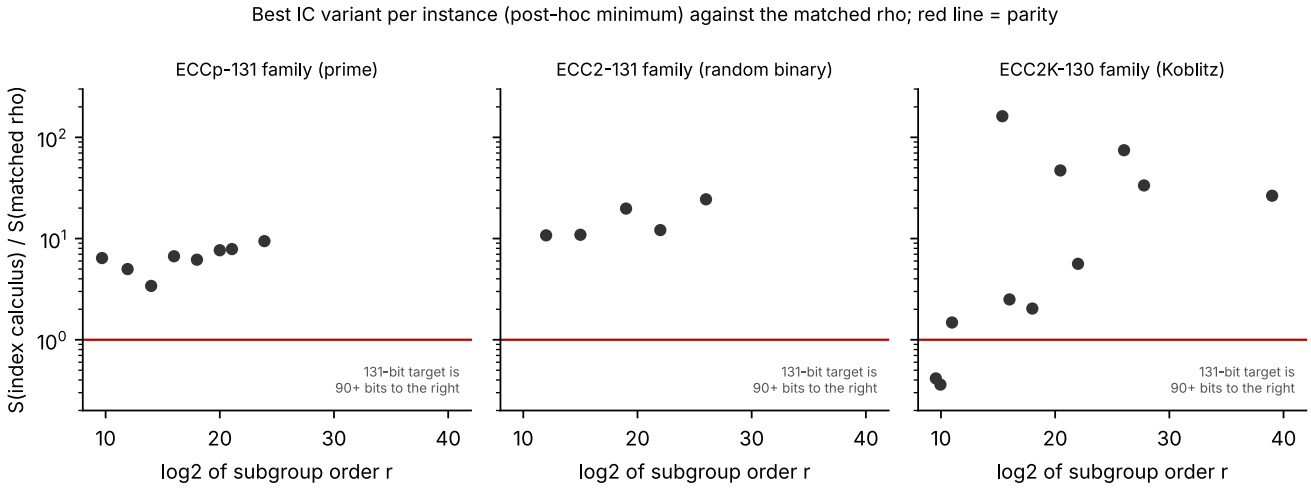


Figure 3: S(IC) / S(matched rho) from `raw/ic_boundary_default.json` (`ic boundary --seed 131`, 24 instances, all verified, 3 repeats, 8 rho repeats). The plotted variant per instance is the minimum over variants, chosen after the fact; red line is parity. The three target curves sit at 130 to 131 bits, off the right edge.

Per-regime read-out (measured, best variant per instance; S in group-addition equivalents per square root of r). Prime: ratio 6.4, 5.0, 3.4, 6.7, 6.2, 7.7, 7.9, 9.4 at 2^9.7 to 2^23.9. Random binary: 10.8, 10.9, 19.8, 12.1, 24.4 at 2^12 to 2^26. Koblitz: 0.41, 0.36, 1.48 at 2^9.6 to 2^11 (rho setup dominates there), then 161.7 (an outlier at 2^15.4), 2.5, 2.0, 47.1, 5.6, 74.6, 33.5, 26.5 at 2^16 to 2^39. Fitted total-operations exponents (the tool's own, with R2): prime balanced walk 0.451 against rho 0.371; random binary balanced walk 0.486 against rho 0.399; Koblitz folded walk 0.459 (R2 0.82) against rho 0.253. Over these ranges IC grows faster than rho in every regime. Extrapolating to 2^131 from these exponents is an extrapolation, not a measurement, and I do not give a number: the rho fit exponents sit well below the theoretical 0.5, so the fits are dominated by setup and range effects.

ECCp-131 exact-curve probe (measured).

arm	pair sums or table	median ms (5 runs)	natural relations	rank	log2 hit bound per candidate
2-point B256	65,536	76.6	0	0	−113.2
2-point B512	262,144	127.5	0	0	−111.2
2-point B512 (A/A copy)	262,144	111.1	0	0	−111.2
2-point B1024	1,048,576	266.3	0	0	−109.2
3-point B128	16,384 pair table	139.5	0	0	−106.2

Table 4: `ic prime --curve eccp-131 --setup-candidates 32 --target-candidates 16 --seed 131`, in-process `elapsed_ms`. The hit bound is derived: log2(2B^2 or 8B^3) minus log2(n-1), the repository's coverage bound for these oracles. The A/A pair differs by 13%, so only the B256 to B1024 trend, not any smaller gap, is read as real.

For ECC2K-130 and ECC2-131 there is no exact-curve IC run: `icx estimate` puts both in the "scaled" regime (full-width runs use a same-family analogue), and `ic list` has no ECC2-131 profile. The heuristic `icx` model (Kousidis-Wiemers) is a model, not a measurement.

What this study does not show

- No GPU measurement: the host has no GPU, and `gpu/certicom131/` has no CUDA device replay or persistent worker. The ECC2-131 and ECCp-131 rows are single-core CPU rates of unoptimised walks; the ECCp-131 arm uses the header's correctness-first 32-bit limbs and is expected to be slower than a tuned 64-bit implementation by an amount I did not measure.
- No distinguished-point storage, collision or full-solve run for any target. A step rate times expected steps is a projection, not a solve.
- No claim about IC at 131 bits, and no new IC row: the validation gate (replay certificate, validator report, matched rho with the section 5a band) is not met, so nothing is added to `benchmarks/README.md` or the leaderboard.

Next improvements to try (ratchet)

1. A 64-bit-limb Montgomery or special-prime arm for ECCp-131 under the same gates (the largest identified gap in the table above).
2. A normal-basis or bitsliced ECC2-131 walk without Frobenius, to separate field cost from the orbit trick in the 25.2x CPU gap to the ECC2K client.
3. Replay `gpu/certicom131/test_cuda.cu` and port the batched walk on the first approved CUDA runner, then re-read the 15.2x and 21x claims against measured GPU steps.
4. An IC relation oracle that escapes the 8 B^3 support limit on ECCp-131, using the probes here as controls.

Files

`bench131.cpp`, `run_all.sh` (frozen command list), `analyze.py`, `results.json`, `fig1_rho_step.svg/png`, `fig2_projection.svg/png`, `fig3_ic_vs_rho.svg/png`, `raw/` (48 isolation records, 25 probe outputs, audits, ladder JSON, stdout and stderr of every command, `exit_codes.txt`).